P/NP CLASS

Algorithms with polynomial time complexity
$O(n^k)$
→ $O(n)$
→ $O(n^2)$
→ $O(n \log n)$ ...

Algorithms with non-polynomial time complexity [non-deterministic polynomial time complexity]
→ 2^n
→ $n!$

Optimisation algorithms/problems come under NP Class

BASICS of COMPUTATIONAL COMPLEXITY

COMPUTABLE PROBLEM

A computable problem implies that the problem has an algorithm with which it can be solved.

NON-COMPUTABLE PROBLEM

A non-computable problem implies that the problem has no algorithm.

HARD and EASY TERMS

They are used to refer to the problems that can't be computed and are computable, respectively.

UPPER BOUND

It is the measure of the maximum computational effort (worst case) required to solve the given problem.

WORST CASE TIME COMPLEXITY

It refers to the maximum amount of time taken by the algorithm to produce the correct output for any valid input.

Upper Bound in Search: O(n)
in Sort :

LOWER BOUND

It is defined as the minimum amount of work needed to solve a problem. It indicates that the given algorithm will go no less than the lower bound specified resource.

A rule of thumb is that if the complexity of an algorithm equals the lower bound, said algorithm is optimal.

A lower bound is also called tight bound if the best algorithm's run time complexity is the same as the lower bound.

DECISION PROBLEM / RECOGNITION PROBLEM

A problem in which the output is restricted to yes or no.

P-CLASS

All the problems (decision/optimisation) for which polynomial time algorithms exist are called P-Class problems. These problems are considered as easy problems/tractable problems.

NP-CLASS

This consists of decision problems for which there exists a verification algorithm that can verify the input decision problem in polynomial time.

NP is a class of language L that can be verified by an algorithm in polynomial time.

These problems are intractable problems.

TYPES of NP PROBLEMS

→ NP HARD
→ NP COMPLETE

NP-HARD

They are similar to any other problems under class NP.

NP - COMPLETE

They are the hardest of class NP problems that are in between NP Hard and P classes.

To prove that a language is in NP-Complete,

1) A language is NP-Complete if L is an element of NP.
2) L' reduces to L polynomially for every L' that is an element of NP.

1) $L \in NP$
2) $L' \leq_p L$ for every $L' \in NP$

NP COMPLETENESS (NPC)

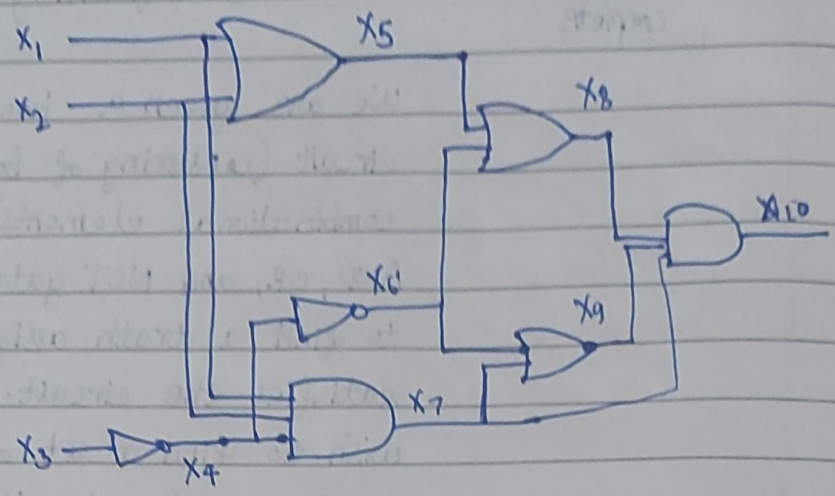
First NP Complete Problem: Circuit Satisfiability Problem

We are given a boolean combinational circuit (consisting of boolean combinational elements) consisting of AND, OR, and NOT gates. We wish to find a truth assignment that satisfies the circuit. That is, we wish to find a set of boolean inputs to the circuit that causes its output to be 1.

Language $L \subseteq \{0,1\}$ is NPC if

1) $L \in NP$
2) $L' \leq_p L$ for every $L' \in NP$

FORMULA SATISFIABILITY (SAT)



$x_5 = x_1 \vee x_2$

$x_4 = \sim x_3$

$x_6 = \sim x_4$

$x_7 = x_1 \wedge x_2 \wedge x_4$

$x_8 = x_5 \vee x_6$

$x_9 = x_6 \vee x_7$

$x_{10} = x_8 \wedge x_9 \wedge x_7$

$x_{10} = (x_1 \wedge x_2 \wedge x_4) \wedge (x_5 \vee x_6) \wedge (x_6 \vee x_7)$

$x_{10} = (x_1 \wedge x_2 \wedge \sim x_3) \wedge (x_1 \vee x_2 \vee \sim x_4) \wedge (\sim x_4 \vee (x_1 \wedge x_2 \wedge x_4))$

$x_{10} = (x_1 \wedge x_2 \wedge \sim x_3) \wedge (x_1 \vee x_2 \vee x_3) \wedge (x_3 \vee (x_1 \wedge x_2 \wedge \sim x_3))$

POSSIBLE TRUTH ASSIGNMENT that SATISFIES

$\Rightarrow (1, 1, 0) \rightarrow$ certificate

$\downarrow \; \downarrow \; \downarrow$

$x_1 \; x_2 \; x_3$

(can be verified in polynomial time)

So, $L \in NP$.

SAT is NP. It's certificate can be verified in polynomial time. First half is verified.

Second Half: $L' \leq_p L$

L' reduces polynomially to L

T.P.: $Ckt\text{-}SAT \leq_p SAT$

Circuit Satisfiability — polynomially reduces to — Formula Satisfiability

Formula ϕ produced by the reduction algorithm is the AND of the circuit output variable with the conjunction of clauses describing the operation of each gate.

For the above circuit, the formula is

$$\begin{aligned}\phi = x_{10} &\wedge (x_4 \leftrightarrow \neg x_3)\\ &\wedge (x_5 \leftrightarrow x_1 \vee x_2)\\ &\wedge (x_6 \leftrightarrow \neg x_4)\\ &\wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4))\\ &\wedge (x_8 \leftrightarrow (x_5 \vee x_6))\\ &\wedge (x_9 \leftrightarrow (x_6 \vee x_7))\\ &\wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))\end{aligned}$$

For (1,1,1),

$x_4 = 0$ $\quad x_8 = 1$

$x_5 = 1$ $\quad x_9 = 1$

$x_6 = 1$ $\quad x_{10} = 0$

$x_7 = 0$

$$\begin{aligned}\phi &= 0 \wedge (0 \leftrightarrow 0) \wedge (1 \leftrightarrow 1)\\ &\wedge (1 \leftrightarrow 1) \wedge (0 \leftrightarrow 0)\\ &\wedge (1 \leftrightarrow 1) \wedge (1 \leftrightarrow 1) \wedge (0 \leftrightarrow 0)\end{aligned}$$

$\phi = 0 \wedge 1 \wedge 1 \wedge 1 \wedge 1 \wedge 1 \wedge 1 \wedge 1$

$\phi = 0$

Given a circuit C, it is straight forward to produce such a formula ϕ in p-time.

If C has a satisfying assignment, each wire of the circuit has a well-defined value, and the output of the circuit is 1. Therefore, the assignment of wire values to variables in ϕ makes each clause of ϕ evaluate to 1, and thus the conjunction of all evaluates to 1. Conversely, if there's an assignment that causes ϕ to evaluate to 1, the circuit C is satisfiable.

Hence, circuit-SAT $\leq_p$ SAT.

3-CNF SATISFIABILITY

3CNF → AND of ORS
↳ 3 distinct literals
Eg: $(x_1 \vee \sim x_4 \vee \sim x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\sim x_4 \vee \sim x_3 \vee \sim x_4)$

Prove that satisfiability of boolean formulas in 3-CNF is NP-Complete.

TP:

1. 3CNF-SAT $\in$ NP
2. SAT $\leq_p$ 3CNF-SAT

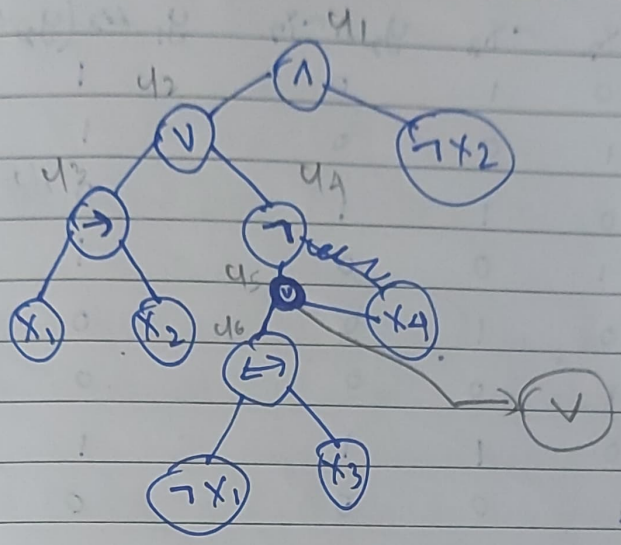
(1) As SAT $\in$ NP, 3-CNF-SAT $\in$ NP.
→ Certificate can be verified in p-time
→ Finding a solution takes exponential or super polynomial time.

② Reduction of SAT to 3-CNF-SAT

- 3 steps - each step transforms input formula ϕ closer to desired 3-CNF form

$$\phi = (x_1 \to x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \sim x_2$$

Step 1 Construct Binary Parse Tree



Rewritten ϕ,

$$\begin{aligned}\phi' = y_1 &\wedge (y_1 \leftrightarrow (y_2 \wedge \sim x_2)) \\ &\wedge (y_2 \leftrightarrow (y_3 \vee \neg y_4)) \\ &\wedge (y_3 \leftrightarrow (x_1 \to x_2)) \\ &\wedge (y_4 \leftrightarrow \neg y_5) \\ &\wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ &\wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))\end{aligned}$$

Here, ϕ' is the conjunction of clauses ϕ_i', each of which has atmost 3 literals. The only additional requirement is that each clause be an OR of literals.

Step 2 Convert each ϕ_i' into CNF Form.

→ Truth Table

→ DNF

→ Convert to CNF (De Morgan's Law)

Eg: $y_1 \leftrightarrow (y_2 \wedge \sim x_2)$

y_1	y_2	x_2	$\sim x_2$	$y_2 \wedge \sim x_2$	$y_1 \leftrightarrow (y_2 \wedge \sim x_2)$
0	0	0	1	0	1
0	0	1	0	0	1
0	1	0	1	1	0
0	1	1	0	0	1
1	0	0	1	0	0
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	0	0	0

DNF:

↓
disjunction
of max terms

De Morgan's Law to get CNF:

Step 3 Ensure each clause has exactly 3 distinct literals.

→ If yes, include in ϕ_i

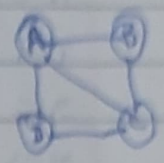
→ If 2, include missing literal $(l_1 \vee l_2 \vee (p \wedge \neg p))$

→ If 1, include both missing literals $(l_1 \vee (p \wedge \neg p) \vee (q \wedge \neg q))$

CLIQUE

→ Undirected Graph $G = (V, E)$
→ Clique : Subset $V' \subseteq V$, each pair is connected by an edge in E.
→ Complete Subgraph of G
→ Size = No. of Vertices

Eg.



ABC → clique
ABD → not a clique

CLIQUE PROBLEM

→ Finding clique of maximum size
→ T.P clique $\in NP$, ask if a clique of given size k exists in a graph?
→ CLIQUE = $\{\langle G, k\rangle$: G is a graph with a clique of size k$\}$

① CLIQUE is NP COMPLETE → CLIQUE $\in$ NP

For a given graph $G = (V, E)$, use the set $V' \subseteq V$ of vertices in the clique as a certificate for G. Checking if V' is a clique can be accomplished in polynomial time, by checking if for each pair $(u, v) \in V'$, the edge (u, v) belongs to E.

② CLIQUE is NP HARD → 3CNF-SAT $\leq_p$ CLIQUE

$\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ → boolean formula in 3-CNF, k clauses
Each clause has 3 distinct literals l_1^r, l_2^r, l_3^r.
Construct $G = V, E$ as follows → Put an edge btwn 2 vertices v_i^r, v_j^s if
→ v_i^r, v_j^s are in different triplets
→ v_i^r is not the negation of v_j^s (corresponding literals are consistent)

Eg:

$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$

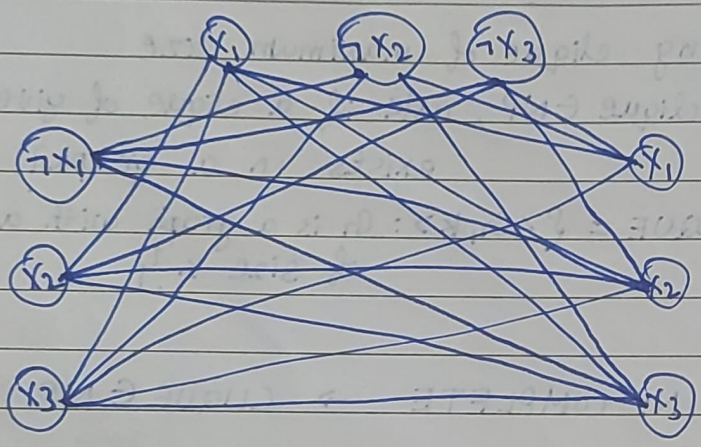
$k=3$ clauses

$r=1$ $C1 \Rightarrow l_1^1, l_2^1, l_3^1$

$r=2$ $C2 \Rightarrow l_1^2, l_2^2, l_3^2$

$r=3$ $C3 \Rightarrow l_1^3, l_2^3, l_3^3$

A graph can be constructed such that ϕ is satisfiable iff G' has a clique of size k.



This transformation of ϕ to G is reduction. If ϕ has a satisfying assignment, then each clause C_r contains at least one literal l_i^r that is assigned to 1, and each such literal corresponds to a vertex v_i^r.

Picking up one such vertex 'true' literal from each clause yields a set 'V' of 'k' vertices. Then, V is a clique.